CivicLens: Multimodal Classification and Urgency Prioritization of Citizen-Reported Urban Issues

Stage 1: Classifying Citizen Reports from Text on Open Municipal 311 Data

Aron Palamoottil Biju
Student ID: 73706
Bachelor's in Computer Engineering
Faculty of Computer Science, Graphics and Architecture
Vistula University, Warsaw, Poland
apalamo1@stu.vistula.edu.pl

Student Research Challenge, Architecture of Computers (CII3SE003CI), Winter Semester 2026/2027
November 2026

Abstract

Cities receive many citizen reports about problems such as missed rubbish collection, potholes and overgrown lots, and staff still sort most of them by hand. An earlier CivicLens review found that published classifiers use private data and miss small categories. In this study, I tested text classifiers on open data, as the first step towards the multimodal CivicLens system. I used the public-domain Open311 dataset of Bloomington, Indiana (130,228 requests). After removing empty texts, label echoes and duplicates, 79,500 reports in 56 categories remained. I compared a majority baseline, three TF-IDF models (Naive Bayes, Logistic Regression, linear SVM) and a frozen MiniLM sentence encoder with Logistic Regression, each with and without class balancing, using a leakage-controlled, group-aware split. Hyperparameters were tuned by grouped 3-fold cross-validation on the training data only, and models were compared by macro-F1 on 15,924 test reports with 95% cluster-bootstrap confidence intervals. The best models were balanced Logistic Regression (macro-F1 0.499, 95% CI 0.484–0.510) and balanced linear SVM (0.497); the difference between them was not clear. Both were better than the MiniLM model (0.476). Balancing raised macro recall for every model, improved macro-F1 for the three TF-IDF models, but lowered it for MiniLM. Errors were concentrated in overlapping categories such as Trash, Excessive Growth and Yard Waste, and in very small categories. The study gives CivicLens a tested, reproducible text baseline and shows that one third of the texts in this dataset are duplicates, which must be removed before evaluation.

Keywords: citizen reports; 311 service requests; text classification; class imbalance; sentence embeddings; data leakage

1. Introduction

1.1 Global Context

Many cities let people report non-emergency problems through 311 hotlines, web forms and apps. Each report must be read, given a category and sent to the right department. Chen et al. (2019) describe a Chinese service centre where staff dispatched requests to 157 departments by hand. A wrongly routed report has to be sent again, so the repair is delayed. Machine learning (ML) can suggest the category automatically. Linear models on TF-IDF word features are classic text classifiers (Joachims 1998), and pretrained sentence encoders turn a sentence into a vector that captures its meaning (Reimers and Gurevych 2019). It is less clear how they work on citizen reports, which are short, informal and very unevenly spread over categories.

1.2 Local Context

The motivating case for CivicLens is Warsaw's contact centre 19115, where residents report problems by phone, web portal or app (Miasto Stołeczne Warszawa n.d.). The long-term aim of CivicLens is to suggest a category and an urgency level for each report. I could not find a public

19115 dataset with report texts, so I used open 311 data from Bloomington, Indiana (USA), which has the same structure: a citizen description and a service category.

1.3 Prior Work and Problem Statement

This paper builds on an earlier CivicLens coursework paper that I wrote with two co-authors (Palamoottil Biju et al. 2026). It was a PRISMA 2020 systematic review (Page et al. 2021) that screened 1,316 records and included eight studies (Appendix B).

Three findings of the review matter here (Appendix Table A1). First, the two text studies used private local data: Chinese hotline requests (Chen et al. 2019) and Indonesian city-app reports (Madyatmadja et al. 2022). Second, class imbalance was not solved. The best model of Chen et al. (2019) reached a micro precision of 0.764 but a macro recall of only 0.352, so most small departments were missed. Madyatmadja et al. (2022) compared five classifiers; a polynomial SVM gave the best F1 (81.83% on four classes), and imbalance was left for future work. Third, the other studies use images (Shama et al. 2024), smartphone sensors (Zareei et al. 2025) or Wikipedia image–text pairs (Poplavskyi and Riabchun 2026). They support the multimodal goal of CivicLens (fusion raised F1 from 80.1% to 86.3%) but give no text baseline. Only Shiraptini et al. (2026), who predict crime hotspots, add explanations (SHAP), and the conceptual studies stress transparency, fairness and privacy (Yan et al. 2023; Zhu and Liu 2025).

So the problem is practical: the reviewed studies give no reproducible comparison of classical and embedding-based classifiers for citizen reports on open data that also tests imbalance handling. Without such a baseline, nobody can say whether the multimodal CivicLens design (Palamoottil Biju et al. 2026) brings a real improvement.

1.4 Research Objectives

My objective was to build and evaluate a reproducible text classifier for citizen reports, as the first tested part of CivicLens. I asked three questions:

- **RQ1.** How do TF-IDF models compare with a frozen MiniLM sentence encoder in macro-F1?
- **RQ2.** Does class balancing improve macro-F1 and the recall of small categories, and what does it cost in accuracy?
- **RQ3.** Which categories are hardest to classify, and what causes the main errors?

Before running the experiment, I stated two hypotheses. H1: the best linear TF-IDF model beats TF-IDF + Naive Bayes, as the SVM did in Madyatmadja et al. (2022). H2: balancing raises macro recall but lowers accuracy. The study is original research with a replication element, because it re-tests the SVM result on another city, language and label set.

1.5 Paper Structure

Sections 2–5 cover the method, results, discussion and conclusion; code and extra tables are in the appendices.

2. Methodology

2.1 Dataset

I used the Open311 dataset of the City of Bloomington (2026), dataset ID aw6y-t4ix, released under the PDDL 1.0 public-domain licence. I downloaded it on 5 November 2026; its SHA-256 checksum is in Appendix C. Only two fields were used: *description*, the text written by the citizen or call-taker, and *service_name*, the category. Address and coordinates were not used. Table 1 shows the statistics that my script computed from the downloaded file.

Table 1. Dataset and preprocessing statistics (computed from the downloaded file)

Step	Reports
Requests in the file (67 categories, no missing labels)	130,228
With a non-empty description after cleaning	120,917
Removed: text only repeats its category name	124
Removed by deduplication (incl. 319 texts with tied labels)	41,293
Final dataset (56 categories)	79,500
Training set / test set	63,576 / 15,924

The data are very imbalanced. In the final dataset, Trash alone has 17,950 reports, while the smallest categories have 51. Many texts are copied templates: one sentence about unsorted recycling appears 2,938 times in the raw file.

2.2 Preprocessing and Leakage Control

If the same sentence is in both the training and the test set, the model only has to remember it, and the score becomes too high (data leakage). The rules below use only a report's text or label (Figure 1, Appendix A):

1. Rows without text are removed. Text is lower-cased, and URLs, e-mail addresses and digit runs of four or more characters (phone numbers, case IDs) are deleted.
2. Texts that only repeat their category name are removed, because they give away the label.
3. Identical texts are kept once, with their most frequent label. In the data, 657 texts had more than one label; 319 of them had a tie and were removed.
4. Near-duplicates are grouped: texts with the same first six words after removing all digits (for example one template with different house numbers) form one group. 9,616 reports were in such groups, and the largest had 331.
5. Categories with fewer than 50 reports (11 categories, 174 reports) are merged into the city's existing Other category, so no report is lost.

I split the data once with StratifiedGroupKFold (five folds, seed 42), ensuring that each group remained entirely within either the training or test set, and used the first fold as the test set (20.0%). All 56 categories appear in the test set, and two checks in the code confirmed that no text or near-duplicate group was shared.

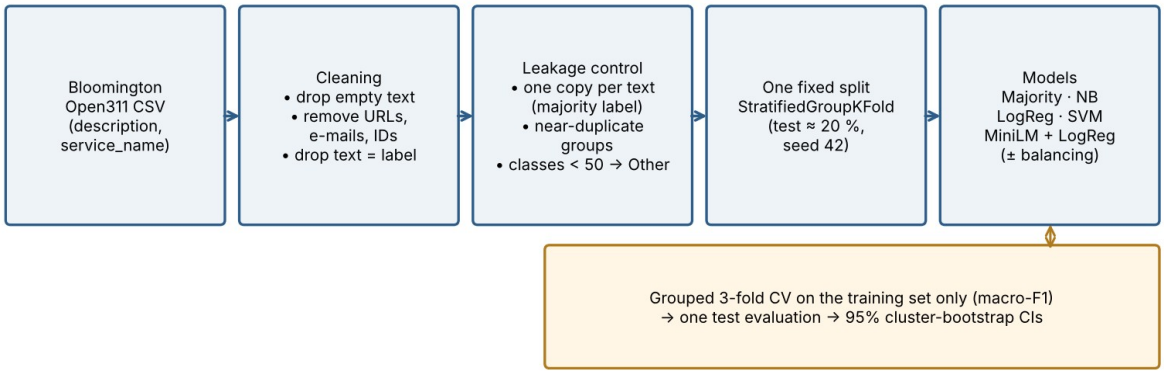


Figure 1. Experimental pipeline. Steps that learn from data use training data only.

2.3 Models

- **Majority baseline:** always predicts the largest training category (Trash).
- **TF-IDF + Naive Bayes (NB)**, **TF-IDF + Logistic Regression (LogReg)** and **TF-IDF + linear SVM** (Joachims 1998).
- **MiniLM + LogReg:** the pretrained all-MiniLM-L6-v2 encoder (Sentence-Transformers n.d.; Wang et al. 2020; Reimers and Gurevych 2019) turns each text into 384 numbers. The encoder is frozen: its weights are not changed and it never sees the labels, so encoding the test texts does not leak information. Only the LogReg is trained.

TF-IDF (term frequency–inverse document frequency) gives a word a high weight when it is frequent in a report but rare in the whole collection. I used single words and word pairs with at most 100,000 terms. The vectoriser sits inside a scikit-learn Pipeline, so it is fitted on training folds only.

Every learning model was trained twice: unweighted and balanced. Balancing gives each training report of category c the weight $N / (K \cdot N_c)$, where N is the number of training reports, K the number of categories and N_c the size of c (He and Garcia 2009), so every category has the same total weight. LogReg and the SVM use class_weight='balanced'; Naive Bayes gets the same weights as sample weights. The searched values are in Appendix Table A2.

2.4 Training and Evaluation

Hyperparameters were chosen by grouped, stratified 3-fold cross-validation (CV) on the training set, using macro-F1. The best setting was refitted on the whole training set and tested once. I used macro-F1 as the main metric because it gives every category the same importance:

$$\textit{Macro-F1} = (1/K) \cdot \Sigma k \, [\, 2 \cdot P_k \cdot R_k / (P_k + R_k) \,]$$

Here P_k and R_k are the precision and recall of category k. I also report accuracy, macro precision, macro recall and weighted F1.

2.5 Statistical Analysis

Since there is one test set, I measured uncertainty with a cluster bootstrap. Whole near-duplicate groups of the test set (14,469 groups) were resampled 2,000 times, and the 2.5th and 97.5th percentiles gave a 95% confidence interval (CI). Models were compared on the same resamples (paired bootstrap). I fixed the decision rule in advance: a difference is clear only if its 95% CI does not include zero. The intervals reflect sampling variation of this test set only, not other splits or cities.

2.6 Ethics and Implementation

The dataset is public, and the model is meant to suggest a category to an operator, not to decide alone. It ran on Google Colab with scikit-learn 1.6.1 (Pedregosa et al. 2011) and sentence-transformers 5.7.0 (Appendix C).

3. Results

3.1 Model Comparison (RQ1)

Table 2 and Figure 2 show the test results. The majority baseline reached an accuracy of 0.231 but a macro-F1 of only 0.007, because it gets 55 of 56 categories wrong. All learning models were far above it. For every model, the CV score was close to the test score (differences of at most 0.03), so cross-validation was a good guide to test performance.

Table 2. Test results on 15,924 reports (macro-F1 with 95% cluster-bootstrap CI)

Model	CV macro-F1	Test macro-F1 [95% CI]	Accuracy	Macro recall
Majority baseline	0.007	0.007 [0.006, 0.007]	0.231	0.018
TF-IDF + NB	0.412	0.423 [0.411, 0.432]	0.666	0.407
TF-IDF + NB (balanced)	0.442	0.457 [0.443, 0.469]	0.616	0.518
TF-IDF + LogReg	0.440	0.461 [0.446, 0.472]	0.695	0.434
TF-IDF + LogReg (balanced)	0.487	**0.499 [0.484, 0.510]**	0.685	0.517
TF-IDF + linear SVM	0.456	0.485 [0.469, 0.497]	0.707	0.467
TF-IDF + linear SVM (balanced)	0.489	**0.497 [0.483, 0.508]**	0.694	0.540
MiniLM + LogReg	0.463	0.476 [0.461, 0.488]	0.679	0.466
MiniLM + LogReg (balanced)	0.433	0.443 [0.432, 0.453]	0.596	0.552

Macro precision and weighted F1 are in model_comparison.csv (supplementary results).

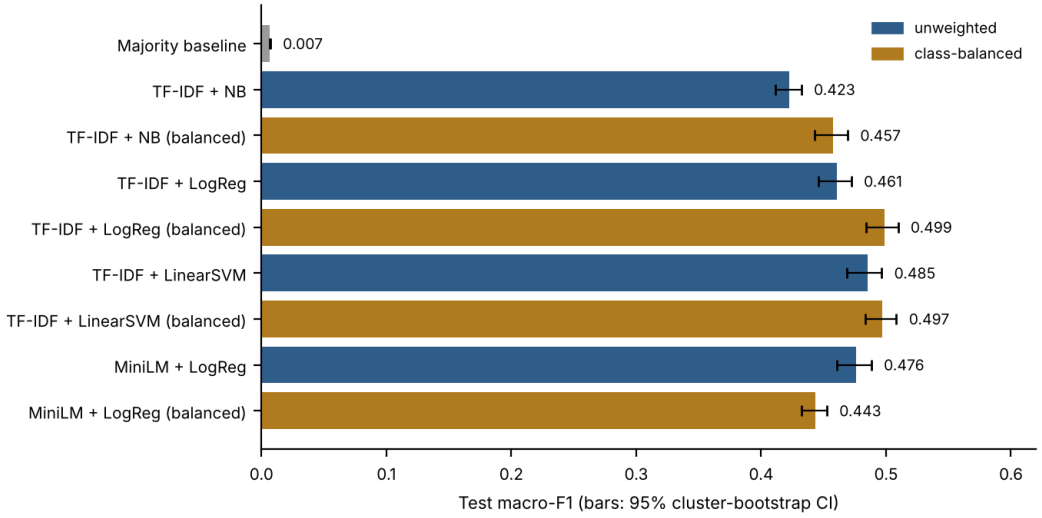


Figure 2. Test macro-F1 of all models with 95% confidence intervals.

The two best models were balanced LogReg (0.499) and balanced SVM (0.497). Their difference, 0.002 (95% CI −0.006 to 0.009), is not clear. Balanced LogReg was clearly better than the best MiniLM model, by 0.022 (CI 0.010 to 0.036). Without balancing, the SVM and MiniLM did not differ clearly (0.009, CI −0.006 to 0.023). Every linear TF-IDF model beat Naive Bayes with the same weighting: for example, the SVM by 0.062 (CI 0.049 to 0.074) without balancing and by 0.039 (CI 0.029 to 0.050) with balancing.

3.2 Effect of Class Balancing (RQ2)

Table 3 shows the effect of balancing. Macro recall went up and accuracy went down for all four model types. Macro-F1 improved clearly for the three TF-IDF models but fell clearly for MiniLM. In a post-hoc analysis of the ten smallest test categories (8–31 reports each), balancing raised the average recall from 0.081 to 0.214 for LogReg, from 0.147 to 0.236 for the SVM, from 0.016 to 0.255 for NB and from 0.215 to 0.459 for MiniLM.

Table 3. Change caused by class balancing (balanced minus unweighted)

Model	Δ macro-F1 [95% CI]	Δ macro recall	Δ accuracy
TF-IDF + NB	+0.035 [0.022, 0.047]	+0.112	−0.050
TF-IDF + LogReg	+0.038 [0.027, 0.050]	+0.083	−0.010
TF-IDF + linear SVM	+0.012 [0.002, 0.023]	+0.072	−0.013
MiniLM + LogReg	−0.033 [−0.044, −0.021]	+0.086	−0.083

3.3 Per-Category Results and Errors (RQ3)

For balanced LogReg, the F1-score was 0.80 or higher for seven categories, for example Temporary Signage (0.95), Potholes and Street Repair (0.88), Recycling (0.84) and Trash (0.84). Seven categories had an F1 below 0.20. Three of them had an F1 of zero: Utilities Yardwork,

Utilities Construction and Utility Construction Projects, all with 10–25 test reports. Figure 3 shows that most errors happen between related categories. The largest confusions were Trash predicted as Excessive Growth (177 reports), Trash as Yard Waste (165) and Excessive Growth as Trash (146). The Other category had a recall of only 0.35.

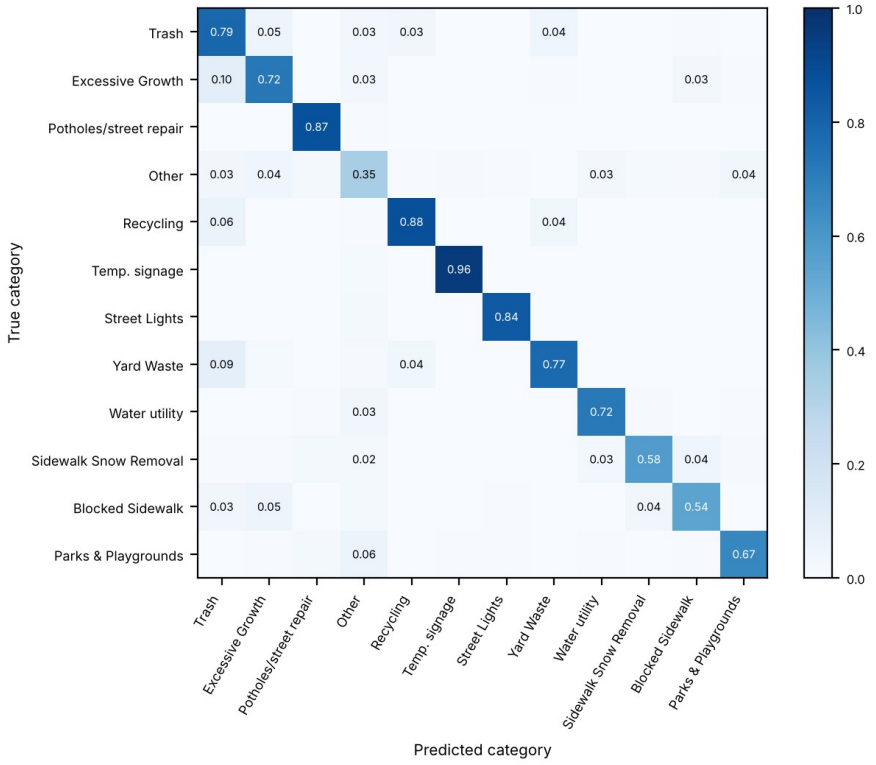


Figure 3. Confusion matrix of balanced LogReg for the 12 largest categories (rows sum to 1 over all 56 categories).

4. Discussion

4.1 Interpretation of Results

RQ1. Balanced TF-IDF LogReg and SVM were clearly better than MiniLM. I think one reason is that 311 reports contain local and service-specific words, such as "sticker", "pick-up" or "brush", which TF-IDF can use directly, while a frozen general encoder may blur them. H1 is supported: both linear TF-IDF models beat Naive Bayes, which agrees with the SVM result of Madyatmadja et al. (2022) on very different data.

RQ2. H2 is supported for all four model types: balancing raised macro recall and lowered accuracy. For the TF-IDF models, F1 rose mainly in the smaller half of the categories (by 0.03–0.08 on average), while the larger half changed little. For MiniLM, F1 fell in both halves although recall rose, and these results do not show why. So balancing should be tested for each model; this is a first answer to the imbalance problem that Chen et al. (2019) left open.

RQ3. The worst errors are not random. Looking at the misclassified texts, I saw three causes. First, some categories overlap: one report often mentions trash, brush and high grass together, but it can have only one label. Second, some categories are almost the same, such as Utilities Construction and Utility Construction Projects. Third, many texts are short staff notes (median 18 words). Also, 657 texts had different labels in the raw data, which shows label noise.

A direct comparison with the literature is not fair, because Madyatmadja et al. (2022) had four classes and Chen et al. (2019) 157 departments. Still, the balanced models reached a macro recall of 0.52–0.55 with 56 categories.

4.2 Practical Implications for CivicLens

For CivicLens, a balanced linear TF-IDF model is a good first component. It is fast (the SVM trained in under two minutes), it needs no GPU, and LogReg gives a probability for each category. Reports with a low probability can go to an operator, and operator corrections can become new training data, the feedback loop the earlier review found missing. However, the probabilities must first be checked for calibration. The confusions also suggest that reviewing overlapping categories could reduce errors. For Polish 19115 reports, a multilingual encoder such as XLM-R (Conneau et al. 2020) could be tested.

4.3 Limitations and Future Research

The study uses one US city, English text and one category scheme, so the results may not transfer to Warsaw. The split is random, not by time, so it does not test how the model works on future reports. To fit the free Colab session, I used 3-fold CV and a small search grid, so a larger search might change the scores slightly. During tuning of balanced Naive Bayes, class weights were computed from the whole training set rather than within each CV fold. This did not involve the test set, but it may have influenced the selected alpha. The six-word grouping key may also miss some near-duplicates. MiniLM was frozen, and fine-tuning might improve it. The analysis of the ten smallest categories was done after the main analysis. The data have no urgency labels, so urgency could not be tested. Future work follows the CivicLens plan: a time-based and cross-city test, a small hand-labelled urgency set, an image branch with a Vision Transformer (Dosovitskiy et al. 2021), and explanations such as important words or Grad-CAM maps (Selvaraju et al. 2017).

5. Conclusion

In this paper, I moved CivicLens from a literature review to an experiment. I compared nine text classifiers on 79,500 real citizen reports from Bloomington's open 311 data, with leakage control and confidence intervals. Balanced TF-IDF Logistic Regression and linear SVM performed best (macro-F1 about 0.50) and were better than a frozen MiniLM encoder. Balancing helped the TF-IDF models and the small categories, but not MiniLM. Most errors came from overlapping categories and short, noisy texts, and one third of the texts were duplicates that must be removed

for a fair test. The main contribution is a tested and reproducible baseline for CivicLens. The next steps are a time-based test, a second city and urgency prediction.

Prior Work

The literature review summarised in Section 1.3 and Appendix B was prepared earlier as part of the Machine Learning course (Palamoottil Biju et al. 2026). The experiment and its analysis are new to this paper.

Data and Code Availability

The code, raw outputs and reproduction steps are available at https://github.com/vistula-team-project-73706/CivicLens. The dataset is public (Appendix C).

6. References

Chen, Tianen, Jincheng Sun, Hongyi Lin, and Yan Liu. 2019. "Hybrid Machine Learning Models of Classifying Residential Requests for Smart Dispatching." Preprint, arXiv:1912.10546. https://doi.org/10.48550/arXiv.1912.10546.

City of Bloomington. 2026. *Open311*. Dataset. City of Bloomington Open Data. Accessed November 5, 2026. https://data.bloomington.in.gov/d/aw6y-t4ix.

Conneau, Alexis, Kartikay Khandelwal, Naman Goyal, et al. 2020. "Unsupervised Cross-Lingual Representation Learning at Scale." In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, 8440–51. Association for Computational Linguistics. https://doi.org/10.18653/v1/2020.acl-main.747.

Dosovitskiy, Alexey, Lucas Beyer, Alexander Kolesnikov, et al. 2021. "An Image Is Worth 16x16 Words: Transformers for Image Recognition at Scale." Paper presented at the International Conference on Learning Representations.

He, Haibo, and Edwardo A. Garcia. 2009. "Learning from Imbalanced Data." *IEEE Transactions on Knowledge and Data Engineering* 21 (9): 1263–84. https://doi.org/10.1109/TKDE.2008.239.

Joachims, Thorsten. 1998. "Text Categorization with Support Vector Machines: Learning with Many Relevant Features." In *Machine Learning: ECML-98*, edited by Claire Nédellec and Céline Rouveirol, 137–42. Lecture Notes in Computer Science 1398. Springer. https://doi.org/10.1007/BFb0026683.

Madyatmadja, Evaristus Didik, Bernardo Nugroho Yahya, and Cristofer Wijaya. 2022. "Contextual Text Analytics Framework for Citizen Report Classification: A Case Study Using the Indonesian Language." *IEEE Access* 10: 31432–44. https://doi.org/10.1109/ACCESS.2022.3158940.

Miasto Stołeczne Warszawa. n.d. "Aplikacja mobilna Warszawa 19115." Accessed November 5, 2026. https://warszawa19115.pl/o-aplikacji-mobilnej.

Page, Matthew J., Joanne E. McKenzie, Patrick M. Bossuyt, et al. 2021. "The PRISMA 2020 Statement: An Updated Guideline for Reporting Systematic Reviews." *BMJ* 372: n71. https://doi.org/10.1136/bmj.n71.

Palamoottil Biju, Aron, Manuel Vazhakal, and Jishnu Krishna Menon. 2026. "CivicLens: Multimodal Classification and Urgency Prioritization of Citizen-Reported Urban Issues; A PRISMA-Based Systematic Literature Review, Gap Analysis and Proposed Solution." Unpublished coursework paper, Faculty of Computer Science, Graphics and Architecture, Vistula University.

Pedregosa, Fabian, Gaël Varoquaux, Alexandre Gramfort, et al. 2011. "Scikit-Learn: Machine Learning in Python." *Journal of Machine Learning Research* 12: 2825–30.

Poplavskyi, Oleksandr, and Yuliia Riabchun. 2026. "Multimodal Detection of Urban Improvement Indicators in Public Visual-Text Content Using Deep Learning." *Radioelectronic and Computer Systems*, no. 1 (117): 125–37. https://doi.org/10.32620/reks.2026.1.09.

Reimers, Nils, and Iryna Gurevych. 2019. "Sentence-BERT: Sentence Embeddings Using Siamese BERT-Networks." In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, 3982–92. Association for Computational Linguistics. https://doi.org/10.18653/v1/D19-1410.

Selvaraju, Ramprasaath R., Michael Cogswell, Abhishek Das, Ramakrishna Vedantam, Devi Parikh, and Dhruv Batra. 2017. "Grad-CAM: Visual Explanations from Deep Networks via Gradient-Based Localization." In *Proceedings of the IEEE International Conference on Computer Vision*, 618–26. IEEE. https://doi.org/10.1109/ICCV.2017.74.

Sentence-Transformers. n.d. "all-MiniLM-L6-v2." Pretrained model. Hugging Face. Accessed November 5, 2026. https://huggingface.co/sentence-transformers/all-MiniLM-L6-v2.

Shama, Farhatun, Abdul Aziz, and Lamisa Bintee Mizan Deya. 2024. "CitySolution: A Complaining Task Distributive Mobile Application for Smart City Corporation Using Deep Learning." Preprint, arXiv:2410.12882. https://doi.org/10.48550/arXiv.2410.12882.

Shiraptini, T. C., Kavitha Dhanushkodi, S. Sakthipriya, S. Akshaya, and Sherly Alphonse. 2026. "AI and Machine Learning-Enabled Cognitive Digital Twin for Crime Hotspot Detection and Analysis." *IEEE Access* 14: 16792–801. https://doi.org/10.1109/ACCESS.2026.3658944.

Wang, Wenhui, Furu Wei, Li Dong, Hangbo Bao, Nan Yang, and Ming Zhou. 2020. "MiniLM: Deep Self-Attention Distillation for Task-Agnostic Compression of Pre-Trained Transformers." In *Advances in Neural Information Processing Systems 33*, 5776–88. Curran Associates.

Yan, Zhenjun, Ling Jiang, Xiaoli Huang, Lifang Zhang, and Xinxin Zhou. 2023. "Intelligent Urbanism with Artificial Intelligence in Shaping Tomorrow's Smart Cities: Current Developments, Trends, and Future Directions." *Journal of Cloud Computing* 12: 179. https://doi.org/10.1186/s13677-023-00569-6.

Zareei, Mahdi, Carlos Alonzo López Castañeda, Faisal Alanazi, Fausto Granda, and Jesús Arturo Pérez-Díaz. 2025. "Machine Learning Model for Road Anomaly Detection Using Smartphone Accelerometer Data." *IEEE Access* 13: 122841–51. https://doi.org/10.1109/ACCESS.2025.3586812.

Zhu, Dongwen, and Hao Liu. 2025. "City AI: A Strategic Framework for Urban Artificial Intelligence Application and Development." *Urban Informatics* 4: 12. https://doi.org/10.1007/s44212-025-00077-9.

Declaration of Generative AI and AI-Assisted Technologies

The author led the research, defined the topic and objectives, made the final research design decisions, conducted the experiments in Google Colab, evaluated the results, and approved the final manuscript. Claude (Anthropic) and ChatGPT (OpenAI) were used under the author's direction for experiment design, coding and debugging, research clarification, manuscript drafting and editing, and draft review. The author takes full responsibility for the paper and its findings.

7. Appendices

Appendix A. Supplementary Tables and Source Code

Table A1. Text-related evidence from the earlier review (values checked against the original articles)

Study	Data	Method	Reported result	Limitation for this study
Chen et al. (2019)	645,924 hotline records, China, 157 departments	TF-IDF / Word2Vec; NB, MLP, residual CNN	Blind test: micro precision 0.764; macro recall 0.352	Imbalance; private data
Madyatmadja et al. (2022)	6,321 city-app reports, Indonesia, 4 classes	Context-aware preprocessing; n-gram TF-IDF; five classifiers	Best F1 81.83% (polynomial SVM, unigrams)	Imbalance not handled; language-specific rules
Poplavskyi & Riabchun (2026)	Wikipedia image–text pairs (WIT) as a proxy	Image and text encoders with fusion	F1 86.3% (multimodal) vs. 80.1% (text only)	Proxy data; binary task

Table A2. Hyperparameter values searched and selected (3-fold grouped CV, macro-F1)

Model	Values searched	Selected (unweighted / balanced)
TF-IDF + NB	alpha {0.01, 0.1, 1.0}	0.01 / 0.1
TF-IDF + LogReg	C {1, 10}	10 / 10
TF-IDF + linear SVM	C {0.1, 0.5}	0.5 / 0.5
MiniLM + LogReg	C {1, 10}	10 / 1

TF-IDF for all text models: word unigrams and bigrams, sublinear term frequency, min_df = 3, max_df = 0.9, at most 100,000 terms.

The complete script (civiclens_experiment.py) is reproduced below as it was executed (version 3, as recorded in the run metadata file run_info.json: script_version “v3”), and is also supplied as a separate file. The only difference from the executed file is the header comment, which wrongly said “version 2” and has been corrected to “version 3”; no code was changed.

```
"""
CivicLens - text classification experiment on open 311 data (version 3)
Author: Aron Palamoottil Biju (Vistula University, student ID 73706)

Design (matches Section 2 of the paper):
  1. Profile the raw CSV (row counts, missing values, class counts, duplicates).
  2. Clean the text. Remove empty texts and texts that only repeat their label.
  3. Resolve duplicates BEFORE splitting: identical texts are kept once, with
     their most frequent label (texts whose labels tie are removed).
  4. Merge categories with fewer than MIN_CLASS reports into "Other"
     (no report is dropped because of its label).
  5. Near-duplicate groups: texts that share their first 6 words after all
     digits are removed (e.g. one template with different house numbers)
     form one group. A group is never split between train and test.
  6. ONE fixed stratified, group-aware split (~80/20, seed 42). The test set is
     never used for any choice; it is evaluated once per model at the end.
  7. Hyperparameters: stratified group 3-fold CV on the training set, macro-F1.
     (v3: smaller grid and capped vocabulary so the run fits in a free Colab
     session; every finished model is saved at once and the run can resume.)
  8. Every learning model in two versions: unweighted and class-balanced.
  9. Uncertainty: 95% cluster-bootstrap confidence intervals (test groups are
     resampled) and paired comparisons between models (2,000 resamples).

Usage (Google Colab, CPU is enough):
```

```
  !pip install sentence-transformers
  !python civiclens_experiment.py --csv Open311.csv --text description --label service_name
  (add --no_embed to skip the MiniLM model)
"""

import argparse, hashlib, json, os, re, time

import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
from sklearn.dummy import DummyClassifier
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (accuracy_score, classification_report, confusion_matrix,
                             f1_score, precision_score, recall_score)
from sklearn.model_selection import GridSearchCV, StratifiedGroupKFold
from sklearn.naive_bayes import MultinomialNB
from sklearn.pipeline import Pipeline
from sklearn.svm import LinearSVC
from sklearn.utils.class_weight import compute_sample_weight

SEED = 42
EMBED_MODEL = "sentence-transformers/all-MiniLM-L6-v2"   # 384-dim, frozen
N_BOOT = 2000
CV_FOLDS = 3


# ------------------------------------------------------------ 1. data
def clean_text(s):
    """Lower-case; remove URLs, e-mails and digit runs of 4+ (phones, IDs)."""
    s = str(s).lower()
    s = re.sub(r"https?://\S+|www\.\S+", " ", s)
    s = re.sub(r"\S+@\S+", " ", s)
    s = re.sub(r"\d{4,}", " ", s)
    s = re.sub(r"[^a-z0-9\s'.,!?-]", " ", s)
    return re.sub(r"\s+", " ", s).strip()


def group_key(text, n_words=6):
    """Near-duplicate key: remove ALL digits, keep the first n_words words."""
    return " ".join(re.sub(r"\d+", " ", text).split()[:n_words])


def profile(raw, text_col, label_col):
    """Statistics of the downloaded file itself (not portal metadata)."""
    desc = raw[text_col]
    empty = desc.isna() | (desc.astype(str).str.strip() == "")
    vc = desc[~empty].astype(str).str.strip().value_counts()
    return {
        "rows": int(len(raw)),
        "missing_or_empty_description": int(empty.sum()),
        "missing_label": int(raw[label_col].isna().sum()),
        "n_categories_raw": int(raw[label_col].nunique()),
        "top10_categories": raw[label_col].value_counts().head(10).to_dict(),
        "most_repeated_description": vc.index[0] if len(vc) else None,
        "most_repeated_description_count": int(vc.iloc[0]) if len(vc) else 0,
        "rows_whose_text_is_repeated": int(vc[vc > 1].sum()),
    }


def prepare(raw, text_col, label_col, min_class, log):
    df = raw[[text_col, label_col]].rename(columns={text_col: "text", label_col: "label"})
    df = df.dropna(subset=["label"])
    df["label"] = df["label"].astype(str).str.strip()
    df["text"] = df["text"].fillna("").map(clean_text)
    df = df[df["text"].str.len() > 0]
    log["after_removing_empty_text"] = len(df)

    same = df["text"] == df["label"].str.lower().map(clean_text)
    log["removed_text_equals_label"] = int(same.sum())
    df = df[~same]

    # Duplicates -> one row per unique text with its most frequent label.
    counts = df.groupby(["text", "label"]).size().rename("n").reset_index()
```

```
    counts = counts.sort_values(["text", "n"], ascending=[True, False])
    top = counts.groupby("text").head(2)
    tie = top.groupby("text")["n"].transform(lambda x: len(x) == 2 and x.iloc[0] == x.iloc[1])
    tied_texts = set(top.loc[tie.astype(bool), "text"])
    log["removed_texts_with_tied_labels"] = len(tied_texts)
    n_conflict = int((df.groupby("text")["label"].nunique() > 1).sum())
    log["unique_texts_with_more_than_one_label"] = n_conflict
    before = len(df)
    df = counts.groupby("text").head(1)
    df = df[~df["text"].isin(tied_texts)][["text", "label"]]
    log["rows_removed_by_deduplication"] = before - len(df)

    # Rare categories -> "Other". Uses only label counts; no text, no model.
    vc = df["label"].value_counts()
    rare = vc[vc < min_class].index
    log["categories_merged_into_Other"] = sorted(rare.tolist())
    log["rows_relabelled_Other"] = int(df["label"].isin(rare).sum())
    df.loc[df["label"].isin(rare), "label"] = "Other"
    df = df.sample(frac=1.0, random_state=SEED).reset_index(drop=True)
    log["final_rows"] = len(df)
    log["final_categories"] = int(df["label"].nunique())
    return df


# ------------------------------------------------------------ 2. models
def tfidf():
    # inside a Pipeline -> fitted on the training folds only
    # unigrams + bigrams, terms in >= 3 training documents, at most 100,000 terms
    return TfidfVectorizer(sublinear_tf=True, ngram_range=(1, 2), min_df=3, max_df=0.9,
max_features=100_000)


def model_list():
    """(name, estimator, grid, input, balanced). 'balanced' means class-balanced
    sample weights for NB (it has no class_weight option) and
    class_weight='balanced' for LogReg and SVM. Both give each class the same
    total weight: n_samples / (n_classes * n_class_samples)."""
    m = [("Majority baseline", DummyClassifier(strategy="most_frequent"), {}, "raw", False)]
    for bal in (False, True):
        tag = " (balanced)" if bal else ""
        cw = "balanced" if bal else None
        m += [
            ("TF-IDF + NB" + tag, Pipeline([("tfidf", tfidf()), ("clf", MultinomialNB())]),
             {"clf__alpha": [0.01, 0.1, 1.0]}, "text", bal),
            ("TF-IDF + LogReg" + tag, Pipeline([("tfidf", tfidf()),
             ("clf", LogisticRegression(max_iter=1000, class_weight=cw, random_state=SEED))]),
             {"clf__C": [1.0, 10.0]}, "text", False),
            ("TF-IDF + LinearSVM" + tag, Pipeline([("tfidf", tfidf()),
             ("clf", LinearSVC(class_weight=cw, random_state=SEED))]),
             {"clf__C": [0.1, 0.5]}, "text", False),
            ("MiniLM + LogReg" + tag, LogisticRegression(max_iter=1000, class_weight=cw,
random_state=SEED),
             {"C": [1.0, 10.0]}, "emb", False),
        ]
    return [x for x in m if "TF-IDF + LogReg" not in x[0]] + [x for x in m if "TF-IDF + LogReg" in
x[0]]   # slow models last


def encode(texts, out_dir):
    """Frozen pretrained encoder: weights are NOT updated and labels are never
    seen, so train and test texts are encoded independently with the same fixed
    function. Mean pooling, L2-normalised, 384 dimensions; texts longer than
    256 word pieces are truncated by the model."""
    key = hashlib.md5("\n".join(texts).encode()).hexdigest()[:12]
    path = f"{out_dir}/emb_{key}.npy"
    if os.path.exists(path):
        return np.load(path)
    from sentence_transformers import SentenceTransformer
    import torch
    enc = SentenceTransformer(EMBED_MODEL, device="cuda" if torch.cuda.is_available() else "cpu")
    X = enc.encode(list(texts), batch_size=128, show_progress_bar=True, normalize_embeddings=True)
    np.save(path, X)
    return X


# ------------------------------------------------------------ 3. metrics
```

```
def scores(y, p):
    return {"accuracy": accuracy_score(y, p),
            "macro_precision": precision_score(y, p, average="macro", zero_division=0),
            "macro_recall": recall_score(y, p, average="macro", zero_division=0),
            "macro_f1": f1_score(y, p, average="macro", zero_division=0),
            "weighted_f1": f1_score(y, p, average="weighted", zero_division=0)}


def bootstrap_idx(groups, rng):
    """Cluster bootstrap: resample whole near-duplicate groups of the test set."""
    members = pd.Series(np.arange(len(groups))).groupby(np.asarray(groups)).apply(np.array).tolist()
    out = []
    for _ in range(N_BOOT):
        pick = rng.integers(0, len(members), len(members))
        out.append(np.concatenate([members[i] for i in pick]))
    return out


def fast_macro_f1(yt, yp, k):
    """Macro-F1 over k integer-coded classes; F1 = 2TP / (n_pred + n_true),
    and 0 for a class absent from both (same as sklearn, zero_division=0)."""
    tp = np.bincount(yt[yt == yp], minlength=k)
    denom = np.bincount(yp, minlength=k) + np.bincount(yt, minlength=k)
    f = np.divide(2 * tp, denom, out=np.zeros(k), where=denom > 0)
    return f.mean()


def ci(y, p, idx, labels):
    k = len(labels)
    v = [fast_macro_f1(y[i], p[i], k) for i in idx]
    return np.percentile(v, [2.5, 97.5])


def paired(y, pa, pb, idx, labels):
    """Difference A - B in macro-F1 on the same resamples; two-sided p-value =
    2 * share of resamples on the 'wrong' side of zero (bounded by 1)."""
    k = len(labels)
    d = np.array([fast_macro_f1(y[i], pa[i], k) - fast_macro_f1(y[i], pb[i], k) for i in idx])
    obs = fast_macro_f1(y, pa, k) - fast_macro_f1(y, pb, k)
    p = min(1.0, 2 * min((d <= 0).mean(), (d >= 0).mean()))
    lo, hi = np.percentile(d, [2.5, 97.5])
    return obs, lo, hi, p


def safe(n):
    return re.sub(r"[^A-Za-z0-9]+", "_", n).strip("_")


# ---------------------------------------------------------- 4. main
def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("--csv", required=True)
    ap.add_argument("--text", default="description")
    ap.add_argument("--label", default="service_name")
    ap.add_argument("--out", default="results")
    ap.add_argument("--min_class", type=int, default=50)
    ap.add_argument("--no_embed", action="store_true")
    a = ap.parse_args()
    for sub in ["", "/per_class", "/confusion", "/figures"]:
        os.makedirs(a.out + sub, exist_ok=True)

    import sklearn, platform
    run_info = {"csv_sha256": hashlib.sha256(open(a.csv, "rb").read()).hexdigest(),
                "csv_modified": time.strftime("%Y-%m-%d %H:%M",
time.localtime(os.path.getmtime(a.csv))),
                "python": platform.python_version(), "pandas": pd.__version__,
                "numpy": np.__version__, "scikit_learn": sklearn.__version__, "seed": SEED, "n_boot":
N_BOOT, "cv_folds": CV_FOLDS, "script_version": "v3"}
    if not a.no_embed:
        import sentence_transformers
        run_info["sentence_transformers"] = sentence_transformers.__version__
    json.dump(run_info, open(f"{a.out}/run_info.json", "w"), indent=2)
    raw = pd.read_csv(a.csv, low_memory=False)
    json.dump(profile(raw, a.text, a.label), open(f"{a.out}/dataset_profile.json", "w"), indent=2,
default=str)
    log = {}
```

```
    df = prepare(raw, a.text, a.label, a.min_class, log)
    df["label"].value_counts().rename("count").to_csv(f"{a.out}/class_distribution.csv")

    # ---- one fixed stratified, group-aware split (first of 5 group folds = test)
    df["group"] = df["text"].map(group_key)
    gsizes = df["group"].value_counts()
    log.update(near_duplicate_groups=int(len(gsizes)), largest_group=int(gsizes.iloc[0]),
               rows_in_multi_row_groups=int(gsizes[gsizes > 1].sum()))
    splitter = StratifiedGroupKFold(n_splits=5, shuffle=True, random_state=SEED)
    tr, te = next(splitter.split(df["text"], df["label"], groups=df["group"]))
    Xtr, Xte = df["text"].values[tr], df["text"].values[te]
    ytr, yte = df["label"].values[tr], df["label"].values[te]
    gtr, gte = df["group"].values[tr], df["group"].values[te]
    assert not set(Xtr) & set(Xte), "leakage: identical text in train and test"
    assert not set(gtr) & set(gte), "leakage: near-duplicate group in train and test"
    missing = sorted(set(df["label"]) - set(yte))
    log.update(train_rows=len(tr), test_rows=len(te), test_share=round(len(te) / len(df), 4),
               categories_missing_from_test=missing)
    json.dump(log, open(f"{a.out}/preprocessing_log.json", "w"), indent=2)
    print(json.dumps(log, indent=2), flush=True)

    Etr = Ete = None
    if not a.no_embed:
        Etr, Ete = encode(list(Xtr), a.out), encode(list(Xte), a.out)

    labels = sorted(df["label"].unique())
    cv = StratifiedGroupKFold(n_splits=CV_FOLDS, shuffle=True, random_state=SEED)
    os.makedirs(f"{a.out}/preds", exist_ok=True)
    rows, preds = [], {}
    for name, est, grid, kind, nb_bal in model_list():
        if kind == "emb" and Etr is None:
            continue
        pfile, rfile = f"{a.out}/preds/{safe(name)}.npy", f"{a.out}/preds/{safe(name)}.json"
        if os.path.exists(pfile) and os.path.exists(rfile):      # resume after a disconnect
            preds[name] = np.load(pfile, allow_pickle=True)
            rows.append(json.load(open(rfile)))
            print(f"{name:30s} loaded from earlier run", flush=True)
            continue
        A, B = (Etr, Ete) if kind == "emb" else (Xtr, Xte)
        if kind == "raw":
            A, B = np.zeros((len(Xtr), 1)), np.zeros((len(Xte), 1))
        fit_kw = {"clf__sample_weight": compute_sample_weight("balanced", ytr)} if nb_bal else {}
        t0 = time.time()
        gs = GridSearchCV(est, grid, scoring="f1_macro", cv=cv, n_jobs=-1, verbose=3)
        gs.fit(A, ytr, groups=gtr, **fit_kw)
        p = gs.predict(B)
        preds[name] = p
        rows.append({"model": name, "cv_macro_f1": round(gs.best_score_, 4),
                     "best_params": json.dumps({k: str(v) for k, v in gs.best_params_.items()}),
                     **{k: round(v, 4) for k, v in scores(yte, p).items()},
                     "fit_seconds": round(time.time() - t0, 1)})
        np.save(pfile, p); json.dump(rows[-1], open(rfile, "w"))
        print(f"{name:30s} test macro-F1 {rows[-1]['macro_f1']:.4f} (CV {rows[-1]['cv_macro_f1']:.4f})
"
              f"{rows[-1]['fit_seconds']:.0f}s", flush=True)

    # ---- uncertainty on the fixed test set
    rng = np.random.default_rng(SEED)
    idx = bootstrap_idx(gte, rng)
    code = {l: i for i, l in enumerate(labels)}
    yte_i = np.array([code[v] for v in yte])
    preds_i = {m: np.array([code[v] for v in p]) for m, p in preds.items()}
    res = pd.DataFrame(rows)
    cis = [ci(yte_i, preds_i[m], idx, labels) for m in res["model"]]
    res["macro_f1_ci_low"] = [round(c[0], 4) for c in cis]
    res["macro_f1_ci_high"] = [round(c[1], 4) for c in cis]
    res.to_csv(f"{a.out}/model_comparison.csv", index=False)

    comp = []
    for base in ["TF-IDF + NB", "TF-IDF + LogReg", "TF-IDF + LinearSVM", "MiniLM + LogReg"]:
        if base in preds and base + " (balanced)" in preds:
            o, lo, hi, p = paired(yte_i, preds_i[base + " (balanced)"], preds_i[base], idx, labels)
            comp.append({"comparison": f"{base}: balanced - unweighted", "delta_macro_f1": o, "ci_low":
lo, "ci_high": hi, "p": p})
    ranked = res[res.model != "Majority baseline"].sort_values("macro_f1", ascending=False)
["model"].tolist()
```

```
    for other in ranked[1:]:
        o, lo, hi, p = paired(yte_i, preds_i[ranked[0]], preds_i[other], idx, labels)
        comp.append({"comparison": f"{ranked[0]} - {other}", "delta_macro_f1": o, "ci_low": lo,
"ci_high": hi, "p": p})
    pd.DataFrame(comp).round(4).to_csv(f"{a.out}/paired_bootstrap.csv", index=False)

    # ---- per-class, confusion matrices, predictions, figures
    for name, p in preds.items():
        pd.DataFrame(classification_report(yte, p, labels=labels, output_dict=True, zero_division=0)).T
\
          .round(4).to_csv(f"{a.out}/per_class/{safe(name)}.csv")
        pd.DataFrame(confusion_matrix(yte, p, labels=labels), index=labels, columns=labels) \
          .to_csv(f"{a.out}/confusion/{safe(name)}.csv")
    pd.DataFrame({"text": Xte, "true": yte, **preds}).to_csv(f"{a.out}/test_predictions.csv",
index=False)

    best = ranked[0]
    cm = confusion_matrix(yte, preds[best], labels=labels, normalize="true")
    fig, ax = plt.subplots(figsize=(0.45 * len(labels) + 3, 0.45 * len(labels) + 2))
    im = ax.imshow(cm, cmap="Blues", vmin=0, vmax=1)
    ax.set_xticks(range(len(labels)), labels, rotation=90, fontsize=7)
    ax.set_yticks(range(len(labels)), labels, fontsize=7)
    ax.set_xlabel("Predicted"); ax.set_ylabel("True"); ax.set_title(f"Row-normalised confusion matrix:
{best}", fontsize=9)
    fig.colorbar(im, fraction=0.04); fig.tight_layout();
fig.savefig(f"{a.out}/figures/confusion_best.png", dpi=200)

    r = res.sort_values("macro_f1")
    fig, ax = plt.subplots(figsize=(7, 0.35 * len(r) + 1))
    ax.barh(r["model"], r["macro_f1"], xerr=[r["macro_f1"] - r["macro_f1_ci_low"],
r["macro_f1_ci_high"] - r["macro_f1"]],
            color="#2F5D8A", capsize=3)
    ax.set_xlabel("Test macro-F1 (95% bootstrap CI)"); fig.tight_layout()
    fig.savefig(f"{a.out}/figures/macro_f1_by_model.png", dpi=200)
    print(res.to_string(index=False))


if __name__ == "__main__":
    main()
```

Appendix B. Protocol of the Earlier Systematic Review

The review was done for the earlier group paper (Palamoottil Biju et al. 2026) and is summarised here for transparency. It followed PRISMA 2020 (Page et al. 2021).

Table B1. Search strings and records per database (from Palamoottil Biju et al. 2026)

Item	Details
Search strings	Q1: multimodal AND urban AND smart city (broadened for arXiv and Scopus); Q2: machine learning AND citizen report; Q3: image and text AND complaint classification
Search dates	31 October – 2 November 2026
Records identified	arXiv 49; IEEE Xplore 490; Springer 150; Scopus 627; total 1,316
Not searched	Web of Science (platform errors) and ACM Digital Library (no institutional access)
Screening	1,277 excluded at title/abstract stage; 39 full texts assessed; 31 excluded; 8 studies included
Inclusion criteria	ML/AI for detecting, classifying or prioritising citizen-reported urban issues; open access; English; journal, conference or preprint

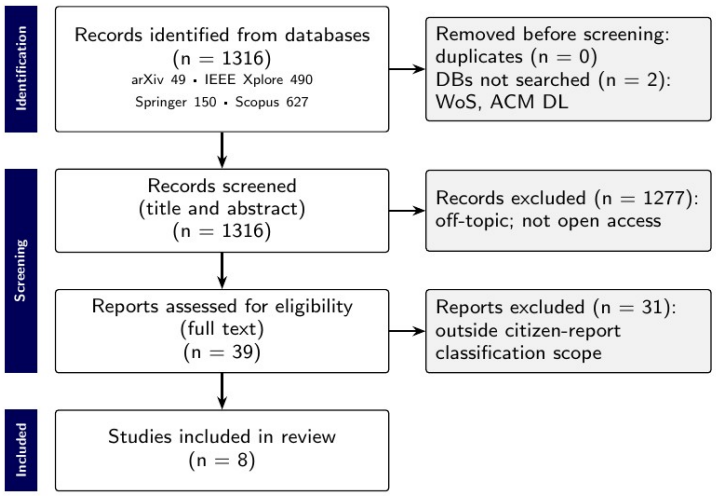


Figure B1. PRISMA 2020 flow diagram of the earlier review (Palamoottil Biju et al. 2026).

Appendix C. Reproducibility

1. Code and results: https://github.com/vistula-team-project-73706/CivicLens (scripts, Colab notebook, requirements.txt, raw outputs, checksums; texts omitted for privacy).
2. Data: https://data.bloomington.in.gov/api/views/aw6y-t4ix/rows.csv?accessType=DOWNLOAD, downloaded on 5 November 2026; SHA-256 802e12249d4e088f3efc813378dd9e1fe126733945ccc2c1b1f814eb1354cc97.
3. Environment: Google Colab with a T4 GPU; Python 3.13.16, pandas 2.2.3, NumPy 2.4.6, scikit-learn 1.6.1, sentence-transformers 5.7.0 (full list in pip_freeze.txt).
4. Command: python civiclens_experiment.py --csv Open311.csv --text description --label service_name --out results
5. Fixed settings: seed 42 for split, folds, models and bootstrap; test set = first of five StratifiedGroupKFold folds; near-duplicate key = first six words without digits; grouped 3-fold CV; 2,000 cluster-bootstrap resamples; categories under 50 reports merged into Other; encoder all-MiniLM-L6-v2 (frozen).
6. Run notes: the run was interrupted once and resumed; finished models were loaded from saved predictions, and the two slowest models were trained last. Because the split and all seeds are fixed, this does not change any result. An earlier run with a larger search grid was stopped by a Colab disconnect and is not reported.
7. All numbers in Section 3 come from model_comparison.csv, paired_bootstrap.csv, per_class/∗.csv, confusion/∗.csv and test_predictions.csv. The following were computed afterwards from the saved test predictions, with the same bootstrap procedure where intervals are given: the comparisons SVM vs. NB, LogReg vs. NB and SVM vs. MiniLM (unweighted), balanced SVM vs. balanced NB and balanced SVM vs. MiniLM (extra_paired_bootstrap_posthoc.csv); the recall of the ten smallest categories; and the per-class F1 change for the smaller and larger halves of the categories.